One Ring to Host It All

Fixed-Capacity Circular Realizations of Finite Functional Trajectories

Chris Bartlett, *Independent Researcher*. ORCID: 0009-0001-3323-0131
Version 1.32, 29 October 2026. DOI: 10.5281/zenodo.23755207
Working paper.
Motivated by *Something is happening: Occurrence Theory*, DOI: 10.5281/zenodo.22875288.

Abstract

Can a finite system repeating on a single cycle reproduce, for a specified duration, the behaviour of another deterministic system, even when the visible behaviour appears irreversible?

We formulate this as a circular-sequence problem. Each visible state has a prescribed number of underlying occurrences, and at least a specified number must begin trajectories that follow the visible rule correctly for a given duration. These trajectories may overlap, but every underlying state must belong to one cycle.

We establish necessary and sufficient conditions for such a realisation and give a constructive method based on a compact integer certificate. We then show that existence can be decided in time polynomial in the binary size of the input, using at most n minimum-cost-flow computations for n visible states, without expanding the potentially enormous underlying cycle.

The result provides an exact resource-feasibility criterion for finite-duration effective dynamics within a recurrent reversible host. It establishes mathematical feasibility under specified conditions, without assuming that the required structures arise naturally.

Keywords: circular words; functional graphs; letter multiplicities; integer certificates; total unimodularity; minimum-cost flow; polynomial-time algorithms; reversible dynamics

2020 Mathematics Subject Classification: Primary 68R15, 90C10; Secondary 05C20, 05C21, 68Q25, 90C27, 90C35.

Overview

In the title, the "ring" is a single recurrent cycle of complete underlying states, and what it may "host" is the entire specified collection of finite trajectories. The theorem determines exactly when this is possible within a fixed capacity. It does not claim that every conceivable process can be accommodated by one cycle.

Can a system that repeats forever support a process that, for a specified period, follows a different rule? In particular, can the visible process forget distinctions even though the complete underlying system never does? This paper answers a precise finite version of that question. It specifies which visible state follows which, how many underlying states display each visible state, at least how many valid starting positions are required (wherever they fall on the cycle), and how long the visible rule must hold. It then decides exactly whether those requirements fit on one underlying cycle.

As an example, suppose the visible rule sends both 0 and 1 to 0, and leaves 0 there. It therefore forgets the initial distinction. Now consider five *distinct* underlying states, visited in this order and then repeated (Figure 1):

$$A \longrightarrow B \longrightarrow C \longrightarrow D \longrightarrow E \longrightarrow A,$$

underlying state	A	B	C	D	E
visible label	1	0	0	0	0

The underlying motion is reversible: every state has exactly one predecessor. Starting at A or at B, the first three transitions display

	start	after 1 step	after 2 steps	after 3 steps
A	1	0	0	0
B	0	0	0	0

Both visible histories follow the rule exactly. The four zeros are not one underlying state repeated four times; they are four different states with the same label. The complete system retains distinctions that the visible description omits.

The requirement is that the visible rule hold for three transitions from at least one start showing 1 and at least one start showing 0. Five is the minimum for these requirements. A valid zero-start needs four successive zero-labelled positions. If any of those positions were revisited within these three transitions, the complete circle would already have been traversed without meeting a 1, contradicting the required one-start. Thus four distinct zero positions and at least one one-position are needed. The displayed cycle attains that bound. The visible rule eventually fails when the cycle returns to A. The guarantee concerns the specified starts and duration, not every position and all future time.

A general answer is not immediate. Longer histories can overlap and share underlying positions. Charging a separate block of states to every start can therefore greatly overstate the requirement. Conversely, constructing several separate working loops does not solve a problem that requires *one* loop. Joining them can break the very histories they were meant to provide. Exact inventories also matter: unused labels cannot simply be discarded or extra states added for convenience.

The paper's first main result gives an exact test. Either the inventory fits one cycle of the visible rule evenly, so that the rule holds forever, which is checked directly; or two short lists of whole numbers satisfy four inequalities. These lists count positions where correct prefixes continue and positions where those prefixes end. A constructive proof shows how any valid certificate can be assembled into one circle without damaging its promised histories. The second result

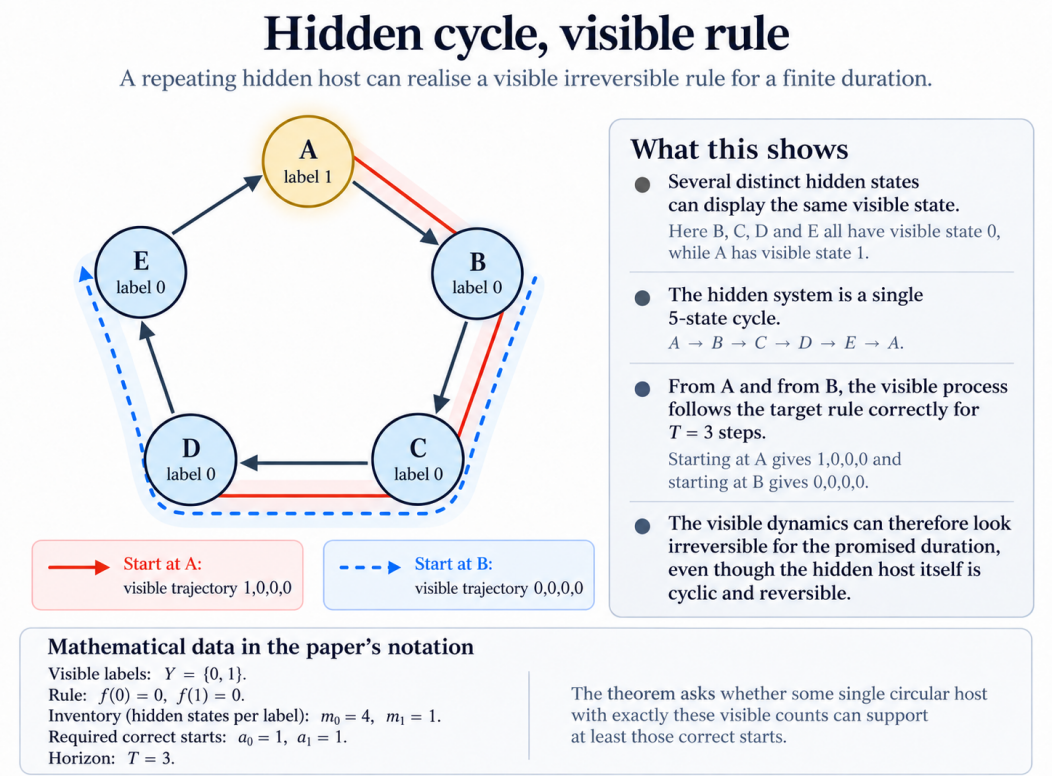


Figure 1: Hidden cycle, visible rule. The five-state example. The hidden host is one cycle $A \to B \to C \to D \to E \to A$; state A shows label 1 and the other four show label 0, so the inventory is $m_0 = 4$, $m_1 = 1$. The visible rule sends both labels to 0, so it forgets whether a history began at 1. Starting at A (solid red) the labels read $1, 0, 0, 0$; starting at B (dashed blue) they read $0, 0, 0, 0$. Both follow the rule for $T = 3$ steps, meeting the demands $a_0 = a_1 = 1$. No hidden state is ever merged with another: the forgetting happens only in the labels.

turns the certificate test into ordinary minimum-cost network-flow problems, with at most as many solves as there are visible labels.

The decision is polynomial in the *length of the input numbers*. A duration with a hundred digits need not be simulated for that many steps before feasibility can be decided. This does not promise that printing an enormous circle is quick, nor that the supplied transparent reference solver has the theoretical worst-case running time. The algorithm returns a compact certificate; expanding the circle takes time appropriate to the size of that output.

A repeating underlying process and finite-duration irreversible visible behaviour need not contradict one another. Here their compatibility has an exact resource test, rather than only an illustrative example. The result can also be used to design constrained periodic sequences or to find the longest duration a fixed inventory can support. These are state-count statements, not bounds on energy, gates, physical volume or how likely nature is to construct the system.

The motivating question arose from *Something is happening: Occurrence Theory* (SiH) [4], which proposes effective worlds supported by underlying occurring processes. This paper develops one finite-duration hosting problem within that programme; accepting its wider metaphysical proposals is not a premise of the proof.

The precise problem and theorem begin in Section 1. Sections 2–5 contain the proofs; Section 6 gives examples and limitations. Readers primarily interested in the motivation and uses can continue directly to Sections 8 and 9. The verification scope is stated in Section 10, the research origin in the AI-assistance statement that follows it, and the conclusion in Section 11.

1. Introduction and problem

Finite trajectories can share long stretches of a deterministic orbit. This makes their circular realisation more economical than allocating a separate word to every required start. Exact inventories introduce a second constraint: every supplied copy of every label must be used, and all copies must lie on one circle. The question studied here is whether these requirements can be decided from compact numerical data, even when the horizon and inventories are too large to expand explicitly. The original motivation arose in an AI mathematical research programme directed by Bartlett to investigate questions inspired by *Something is happening: Occurrence Theory* [4]. Section 8 explains that connection, and Section 9 develops uses and a maximum-horizon consequence. The present problem and its proofs depend only on the finite data below.

Let $Y = \{0, \dots, n-1\}$ be a nonempty finite alphabet, and let $f : Y \to Y$ be a total map. The input data are

$$T \in \mathbb{Z}_{\geq 0}, \qquad a, m \in \mathbb{Z}_{\geq 0}^{Y}, \qquad a \leq m, \qquad N = \sum_{y \in Y} m_y > 0. \tag{1}$$

Vector inequalities throughout are coordinatewise. A host is a circular word $w = (w_0, \dots, w_{N-1})$ with precisely m_y copies of label y. A position i is T-*good* if

$$w_{(i+t) \bmod N} = f^t(w_i) \qquad (0 \leq t \leq T). \tag{2}$$

We require at least a_y distinct T-good positions carrying each label y. These positions are selectable after the word is built; they are not prescribed named positions. Their promised windows may overlap, and the definition permits repeated wrapping around the host. Zero letter counts are allowed, so f need not be restricted to labels present in the host.

Equivalently, the host is one permutation cycle F of length N, with a decoder π whose fibres have sizes m_y; goodness means $\pi(F^t x) = f^t(\pi x)$ for the stated times. No external change to the decoder or host is made during a promised window.

Theorem 1.1 (Fixed-capacity decision)**.** *For every input satisfying* (1)*, existence of a host satisfying* (2) *and the demands a is decidable in time polynomial in the binary input length. Besides a direct exact-cycle check, the decision needs at most n ordinary bounded minimum-cost-flow problems. For a functional component of size q, each flow network has $3q$ vertices and $5q$ arcs before standard supply-demand transformations.*

On a positive decision the algorithm returns either an exact-cycle description or a pair of integer vectors of length n certifying existence. An explicit N-letter host can subsequently be constructed; printing that host is output-sensitive and is not claimed to be polynomial in log N.

The theorem concerns a supplied rule f and specified resources. The standard ingredients are incidence-matrix integrality and polynomial minimum-cost-flow algorithms. The specific modelling work is to retain overlapping occurrences, exact letter counts and one-circle connectivity in a small certificate.

The proof occupies Sections 2–5 and has three parts. Section 2 replaces the search for a circular word of length N by a search for two integer vectors $g, h \in \mathbb{Z}_{\geq 0}^{Y}$: a host exists exactly when either the inventory is a constant exact cycle, which is checked directly, or (g, h) satisfies four families of linear inequalities (Theorem 2.1). The sufficiency half of that proof builds the host, splicing any closed loops into paths so that every position lies on one circle. Section 3 proves an integer-feasibility lemma for box-constrained systems $Ax \leq b$ whose columns record signed

walks in a graph with exactly one cycle, consistently oriented: a real solution implies an integer one, and one bounded minimum-cost-flow problem finds it (Lemma 3.2). Section 4 shows that the first three certificate families have this form on each functional component (Lemma 4.1). It handles the fourth, which requires an endpoint on every target cycle carrying positive demand, by forcing one endpoint at a time (Theorem 4.2). Section 5 combines these tests into the decision algorithm and bounds its running time in the binary input length.

2. An exact integer certificate

Define the push-forward matrix $P = f_*$ by

$$(Pu)_y = \sum_{z:f(z)=y} u_z, \qquad K_T = \sum_{j=0}^{T} P^j.$$

Every weak component of the functional graph of f contains one directed cycle. Write $\mathcal{C}$ for these target cycles, and put

$$\eta_C = \mathbf{1}_{\{\sum_{y\in C} a_y > 0\}}. \tag{3}$$

Only demand on the cycle itself appears in (3); positive demand in its incoming trees does not by itself require a cycle endpoint.

Theorem 2.1 (Single-circle certificate)**.** *Under* (1)*, a required host exists if and only if at least one of the following alternatives holds:*

(i) The positive support of m is exactly one directed cycle of f, and m is constant on that cycle.

(ii) There are $g, h \in \mathbb{Z}^Y_{\geq 0}$ such that

$$g + h \geq a, \tag{4}$$
$$Pg \leq g + h, \tag{5}$$
$$g + K_T h \leq m, \tag{6}$$
$$\sum_{y\in C} h_y \geq \eta_C \qquad (C \in \mathcal{C}). \tag{7}$$

The alternatives need not be disjoint.

Here $g_y + h_y$ counts selected prefix positions of label y. The g_y continuing positions point to another prefix position; the h_y endpoint positions receive a further T positions along the orbit of y. Consequently their total capacity use is

$$(g + h) + \sum_{j=1}^{T} P^j h = g + K_T h.$$

The protected windows may overlap within a prefix. The accounting does not charge a separate disjoint window for each good start.

Proof. Necessity. For any position, (2) is equivalent to correctness of each of the next T physical edges, by induction on t. This equivalence also holds when those edges are visited repeatedly. If every physical edge satisfies $w_{i+1} = f(w_i)$, periodicity implies that the labels follow one directed target cycle. Each of its labels occurs equally often. This is (i), which conversely supplies a word correct at every time.

Otherwise cut the circular word at every incorrect physical edge. The resulting disjoint strings follow f at every internal edge. A string may cross the original index boundary from $N-1$ to 0. A string of length L contains exactly $\max(L-T,0)$ good starts: when $L \geq T+1$ these form its prefix obtained by removing its final T positions. Let p_y count all these prefix positions and let h_y count their last positions. Set $g = p - h$.

The demands give (4). Distinct continuing prefix positions have distinct successors within prefixes, proving (5). The disjoint prefixes and their T-position suffixes give (6). If a prefix contains a demanded position labelled on a target cycle C, all its subsequent labels remain on C; hence its endpoint is on C. This proves (7). The argument includes $T = 0$, when suffixes are empty.

Sufficiency. Alternative (i) was already handled. Given (ii), first remove any unnecessary endpoint-free cycle counts as follows. If h vanishes on a target cycle C, summing (5) over C gives

$$\sum_{\substack{z \notin C \\ f(z) \in C}} g_z \leq 0.$$

Thus each such incoming g_z is zero. The remaining cycle inequalities give $g_{\mathrm{pred}(y)} \leq g_y$ all around C, so g is constant there. The quota implies $a_y = 0$ for every $y \in C$. Setting those g_y to zero therefore preserves all four conditions. Repeat for every target cycle with no endpoint. This step changes only unneeded prefix counts, leaving all suffix counts $K_T h$ in place.

Now create $p_y = g_y + h_y$ distinct prefix positions labelled y and designate h_y of them as endpoints. For each label y, match all continuing positions whose required successor label is y to distinct positions among its p_y slots. Inequality (5) makes these independent labelwise matchings possible. The resulting directed graph has out-degree one at continuing positions, out-degree zero at endpoints, and in-degree at most one everywhere. Its components are paths and directed cycles, including self-loops; an isolated endpoint is a path.

Any internal directed cycle consists of continuing positions and projects onto an entire target cycle C, possibly traversing it more than once. Thus g is nonzero on C. After the deletion step this forces h to be nonzero on C. There is therefore an endpoint v whose label lies on C. Since its out-degree is zero, it lies in a path. Choose an internal-cycle position u with the same label as v, and write its outgoing edge as $u \to u'$. Replace it by $v \to u'$, making u the new endpoint. The replacement respects f, preserves the endpoint counts by label, and joins the entire internal cycle to the path ending at v. It preserves in-degrees and eliminates that internal cycle. The path ending at v now traverses every position of that cycle and ends at u. No suffix has yet been appended. Thus no protected suffix is changed, and the number of internal cycles falls by one at each splice. Repetition leaves only paths.

Append $f(y), \ldots, f^T(y)$ after each endpoint labelled y. If a path has r positions, its ith position ($1 \leq i \leq r$) has $r - i + T \geq T$ correct transitions before the completed block ends. Every prefix position therefore has its entire promised window inside that block. Each nonempty block has length at least $T+1$, and the total length of the blocks is at most N. Thus these windows cannot cross a block join or revisit a physical position. This is compatible with the definition allowing repeated wrapping: the exact-cycle construction handles arbitrary wrapping, while the block construction needs none. Concatenate these path-and-suffix blocks and close them into one circle. The unused copies allowed by (6) may be inserted at block joins, after the protected suffixes. They cannot interrupt any promised window. This gives exactly the capacities m and at least the demands a. If no prefix positions remain, then $a = 0$ and any circle with counts m suffices. Since $N > 0$, that circle is nonempty. □

The splice can be seen in a concrete case. For $f = (1, 0, 0)$, $T = 1$, $a = (2, 2, 0)$ and $m = (3, 3, 1)$, take $g = (2, 2, 0)$ and $h = (0, 1, 0)$. An admissible matching may produce two cycles $0_a \to 1_a \to 0_a$, $0_b \to 1_b \to 0_b$, and the isolated endpoint 1_c. Two same-label splices turn these into

$$1_c \to 0_a \to 1_a \to 0_b \to 1_b.$$

Appending the suffix 0 and placing the unused 2 at the join gives the host $(1, 0, 1, 0, 1, 0, 2)$. Its good-start counts are $(2, 3, 0)$, and its letter counts are exactly m.

Remark 2.2 (Boundary cases). At $T = 0$, $g = 0, h = a$ is always a certificate. If $a = 0$, the zero certificate suffices for every horizon. The inequalities also make sense for $N = 0$, but that auxiliary certificate problem does not describe a nonempty host. If $a \neq 0$ and $T \geq N$, any nonexact host has an incorrect edge within every T-transition window, so only alternative (i) can give a host. The proof of the certificate is consistent with this: after deletion, any nonempty path and its suffix require at least $T + 1 > N$ positions.

3. Integer feasibility on a unicyclic base

We use the incidence convention $+1$ at an arc head and -1 at its tail. A loop has a zero incidence column. A *reduced* incidence matrix has one vertex row deleted.

Lemma 3.1 (Standard bounded-flow integrality)**.** *Let H be a directed incidence matrix, possibly reduced and possibly with loop or parallel columns. With integer balances and finite integer lower and upper bounds, the nonempty polytope of flows satisfying those balances and bounds has integer vertices. Every linear objective has an integer optimum on that polytope.*

Proof. Every square minor of an incidence matrix is $0, 1$ or -1. Indeed, if a square submatrix has a column with at most one nonzero, expand its determinant along that column and use induction. Otherwise every column has a $+1$ and a -1, so its rows are dependent. Appending signed unit rows for coordinate bounds preserves this property. At a vertex, independent tight equations form a nonsingular square subsystem with determinant ± 1 and integer right-hand side. Its solution is integral. Boundedness supplies an optimal vertex for every linear objective. □

This is a standard total-unimodularity argument; it is included to make the integrality dependence explicit. Polynomial algorithms for the ordinary minimum-cost-flow problem are established separately, for example by Orlin [11].

Let G be a connected directed multigraph whose underlying undirected multigraph has exactly one cycle, and suppose that cycle is consistently oriented. Loops and two-edge cycles are permitted. Write B for its reduced incidence matrix: one vertex row is deleted, so its rows are indexed by the remaining vertices and its columns by the base arcs. Then

$$\ker B = \mathbb{R}z, \qquad z_e = \begin{cases} 1, & e \text{ lies on the cycle,} \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

To see this, deleting one cycle arc leaves a spanning tree. Its reduced incidence matrix has determinant ± 1, by leaf elimination. Therefore B has full row rank and a one-dimensional kernel. The displayed nonzero vector z lies in it. In particular, $z \geq 0$.

Let A be any integer matrix whose rows are indexed by the base arcs. Suppose

$$BA = D, \tag{9}$$

where each column of D is the reduced incidence column of an arc on the same vertices; zero columns are allowed. Signed traversal counts of walks in the underlying graph furnish such columns of A. Backwards traversals count negatively and repeated traversals count with multiplicity; their incidences telescope.

Lemma 3.2 (Integer feasibility in an integer box)**.** *Under* (8)–(9)*, let b, ℓ, u be integer vectors, with finite $\ell \le u$. If*

$$Ax \le b, \qquad \ell \le x \le u \tag{10}$$

has a real solution, it has an integer solution. Its feasibility and an integer witness can be found using at most one bounded ordinary minimum-cost-flow problem.

Proof. For each row e, define the integer slack bound

$$U_e = b_e - \min_{\ell \le x \le u} (Ax)_e.$$

The minimum is attained by choosing an appropriate endpoint of each coordinate interval. If some $U_e < 0$, even that row cannot be satisfied. Otherwise every solution of (10) supplies $s = b - Ax$ satisfying $0 \le s \le U$ and the flow equations

$$Dx + Bs = Bb, \qquad \ell \le x \le u, \quad 0 \le s \le U. \tag{11}$$

Because $[D\ B]$ is an incidence matrix, these are ordinary bounded network-flow constraints. Integer lower bounds, including negative ones, can be shifted into the balances.

More generally, any finite nonnegative integer slack bounds that retain the canonical slacks $b - Ax$ of every real feasible point may replace U in the same decision test. The application below uses a simpler common bound of this kind.

For every solution of (11), $B(Ax + s - b) = 0$, so (8) gives

$$Ax + s - b = \lambda z. \tag{12}$$

Choose a cycle arc e_0 and minimise the integer linear flow cost

$$\Phi(x, s) = (Ax)_{e_0} + s_{e_0} = b_{e_0} + \lambda. \tag{13}$$

An original real solution supplies a flow of cost b_{e_0}. Lemma 3.1 supplies an integer optimum with cost at most b_{e_0}. At that optimum $\lambda \le 0$, and its integer vector x already satisfies

$$Ax - b = \lambda z - s \le 0. \tag{14}$$

It remains in the original coordinate box, proving integer feasibility. This implication uses $s \ge 0$ and $z \ge 0$, not the slack upper bounds.

For explicit slack reconstruction, λ is an integer since $z_{e_0} = 1$. Replace s by $s' = s - \lambda z$. This adds a nonnegative integer circulation and gives $Ax + s' = b$. The bounds are preserved: $s' \ge 0$ and $s'_e = b_e - (Ax)_e \le U_e$ by the definition of U_e.

Conversely, an infeasible network or an optimum exceeding b_{e_0} excludes the cost-b_{e_0} flow that any original solution would supply. Thus the stated threshold is an exact decision test. All variables are bounded, so negative costs in the abstract lemma cause no unboundedness. □

Corollary 3.3 (Coordinatewise rounding)**.** *For any real solution x^* of $Ax \le b$ under the lemma's hypotheses, there is an integer solution with $\lfloor x^*_j \rfloor \le x_j \le \lceil x^*_j \rceil$ for every coordinate j. Any integer coordinate bounds already satisfied by x^* are preserved.*

Proof. Apply Lemma 3.2 to that integer box. □

The corollary concerns existence of an integer feasible point. It does not say that every vertex is integral or that arbitrary additional side constraints or objectives are preserved.

An alternative proof uses a spanning-tree decomposition. The same feasibility implication can be seen directly through a totally unimodular extension. Put a cycle arc e last, and write $B = [B_0\ \beta]$, $A = (A_0; w)$ and $z = (z_0; 1)$. Deleting e leaves a spanning tree, so B_0 is unimodular. The identities $BA = D$ and $Bz = 0$ give

$$A = R + zw, \qquad R = \begin{pmatrix} B_0^{-1}D \\ 0 \end{pmatrix}, \qquad z_0 = -B_0^{-1}\beta. \tag{15}$$

The matrix $[B_0^{-1}D\ \ z_0] = B_0^{-1}[D\ \ -\beta]$ is totally unimodular: each square minor is, up to sign, the determinant of the tree incidence matrix with the corresponding tree columns replaced by incidence columns, divided by $\det B_0 = \pm 1$. These determinants are 0 or ± 1 by incidence total unimodularity. Appending the unit row $(0, \dots, 0, 1)$ proves that $[R\ z]$ is totally unimodular. The empty tree matrix for a one-vertex loop is covered by the same convention.

Let τ_- and τ_+ be the minimum and maximum of wx on the integer box $[\ell, u]$. They are integers. The polytope

$$\Pi = \{(x, t) : Rx + zt \le b,\ \ell \le x \le u,\ \tau_- \le t \le \tau_+\}$$

is integral by total unimodularity and integer right-hand sides. A real solution x^* of (10) supplies $(x^*, wx^*) \in \Pi$. Minimise $wx - t$ over Π; an integral optimum $(\hat{x}, \hat{t})$ has value at most zero. Since $z \ge 0$,

$$A\hat{x} = R\hat{x} + z(w\hat{x}) \le R\hat{x} + z\hat{t} \le b.$$

This proves the same integer-feasibility implication. It also isolates the role of consistent orientation: it makes the single kernel direction nonnegative. The bounded-flow proof above supplies the explicit network and cost threshold used by the algorithm.

4. Application to functional trajectories

Work in one functional component of size q, with target cycle C. Initially omit (7) and set $x = (g, h)$. Arrange the three families of inequalities as $Ax \le b$, using row names

$$\begin{array}{rrll} \mathrm{cap}_v: & g_v + (K_T h)_v & \le m_v, & \text{capacity,} \\ \mathrm{mat}_v: & (Pg)_v - g_v - h_v & \le 0, & \text{matching,} \\ \mathrm{dem}_v: & -g_v - h_v & \le -a_v, & \text{demand.} \end{array}$$

For each label v, introduce vertices J_v, K_v, L_v and base arcs

$$\mathrm{cap}_v : J_v \to J_{f(v)}, \qquad \mathrm{mat}_v : J_v \to K_v, \qquad \mathrm{dem}_v : K_v \to L_v. \tag{16}$$

These $3q$ arcs on $3q$ vertices form a connected unicyclic base; its unique cycle is the consistently oriented target cycle among the capacity arcs. The matching and demand arcs attach two-edge twigs.

Lemma 4.1 (Signed-walk representation)**.** *For the reduced base incidence matrix B, $BA = D$, where the columns of D are the incidence columns of the extra arcs*

$$g_v : L_v \to K_{f(v)}, \qquad h_v : L_v \to J_{f^{T+1}(v)}. \tag{17}$$

Proof. The g_v column traverses dem_v, mat_v backwards, followed by cap_v and $\text{mat}_{f(v)}$ forwards. Its coefficient vector is $-\text{dem}_v - \text{mat}_v + \text{cap}_v + \text{mat}_{f(v)}$, exactly the g_v column of A. When $f(v) = v$, the two matching terms cancel, as required. The h_v column traverses dem_v, mat_v backwards and then $T + 1$ successive capacity arcs forwards. Its coefficient on cap_w is the number of $j \in \{0, \dots, T\}$ with $f^j(v) = w$, namely $(K_T)_{wv}$. This remains true with repeated cycle traversals. Each walk telescopes to its endpoints in (17), proving the identity. □

Figure 2 draws the base graph and the extra arcs for a small component.

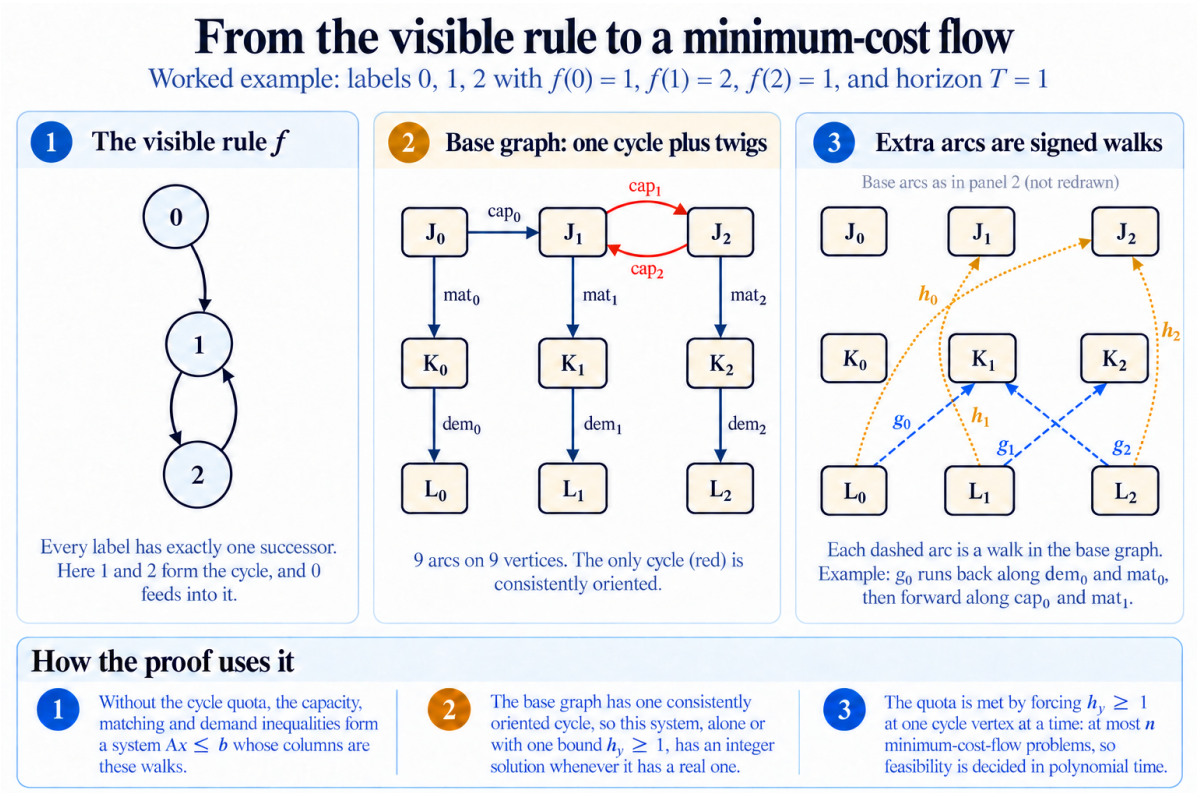


Figure 2: The network of Section 4 for a small example. Labels $0, 1, 2$ with $f(0) = 1$, $f(1) = 2$, $f(2) = 1$ form one functional component ($q = 3$) whose target cycle is $1 \to 2 \to 1$; the horizon is $T = 1$. Panel 2 shows the $3q = 9$ base arcs on $3q = 9$ vertices: $\text{cap}_v \colon J_v \to J_{f(v)}$, $\text{mat}_v \colon J_v \to K_v$ and $\text{dem}_v \colon K_v \to L_v$. Their only cycle, $J_1 \to J_2 \to J_1$, is consistently oriented. Panel 3 adds the $2q = 6$ extra arcs of Lemma 4.1, $g_v \colon L_v \to K_{f(v)}$ and $h_v \colon L_v \to J_{f^{T+1}(v)}$. Each is the net effect of a walk in the base graph: g_0 runs back along dem_0 and mat_0, then forward along cap_0 and mat_1, ending at $K_1 = K_{f(0)}$; h_0 runs back to J_0 and then along $T + 1 = 2$ capacity arcs to $J_2 = J_{f^2(0)}$.

Theorem 4.2 (Forced endpoint). *For nonnegative integer a, m, T on one functional component with $a \le m$, either of the following systems is real-feasible if and only if it is integer-feasible:*

(a) (4)–(6), $g, h \ge 0$, *with no quota;*

(b) the same system together with $h_y \ge 1$, for one specified cycle vertex y.

Each is decidable by one bounded ordinary minimum-cost-flow problem, which returns an integer certificate when feasible.

Proof. The added inequality in (b) is an integer lower bound on one extra arc. Lemma 4.1 allows application of Lemma 3.2. The following explicit common upper bound simplifies the network:

$$U = \max\left(1, \sum_v m_v\right). \tag{18}$$

Use U on all $5q$ variable and slack arcs. Every real feasible point is retained because $K_T \geq I$, hence $0 \leq g_v, h_v \leq g_v + h_v \leq m_v$. The three canonical slack types are

$$m_v - g_v - (K_T h)_v, \quad g_v + h_v - (Pg)_v, \quad g_v + h_v - a_v.$$

Each lies between zero and m_v, so each also respects U.

The network balance equation is $Dx + Bs = Bb$, with $b = (m, 0, -a)$ in the indicated named row order. Choose $v_0 \in C$. Put cost one on g_{v_0} and the slack arc cap_{v_0}, cost $(K_T)_{v_0 v}$ on h_v, and zero on other arcs. The objective is

$$\Phi = g_{v_0} + \sum_v (K_T)_{v_0 v} h_v + s_{\text{cap}_{v_0}} = m_{v_0} + \lambda.$$

A feasible original point has cost m_{v_0}. An integer network optimum with cost at most that value yields the original inequalities by (14). Conversely, a larger minimum or an infeasible network excludes original feasibility. If canonical slacks are desired, the repair $s - \lambda z$ remains within U: the repaired cycle-capacity slack is $m_v - g_v - (K_T h)_v \leq m_v$, and other slacks do not change. The forced endpoint bound is unaffected. Omitting that bound proves (a). □

5. Decision algorithm and binary complexity

The decision algorithm has four steps.

1. Validate (1). If the positive support of m is exactly one target cycle with constant counts, accept and return that exact-cycle description.
2. Find the weak functional components and their target cycles. For a component with $\eta_C = 0$, run the unforced problem of Theorem 4.2. If it fails, reject.
3. For a component with $\eta_C = 1$, test $h_y \geq 1$ for each $y \in C$. Keep the certificate from the first successful phase. If all phases fail, reject.
4. Combine the component certificates and accept. If an expanded host is requested, apply the construction of Theorem 2.1.

Proof of Theorem 1.1. For integer nonnegative h, the cycle quota is equivalent to $h_y \geq 1$ for at least one $y \in C$ whenever $\eta_C = 1$. Theorem 4.2 therefore tests exactly the possible integer certificates in that component. When $\eta_C = 0$ it tests the required unforced system. Different functional components have disjoint variables and constraints, since P and K_T are block diagonal by component. Their endpoint-phase choices are independent. A demanded component needs at most $|C|$ solves; an undemanded component needs one, also at most $|C|$. The total is at most $\sum_{C \in \mathcal{C}} |C| \leq n$. The certificate theorem establishes both the global decision and one-host connectivity. This proves correctness without any assumption about the aggregate real relaxation.

There are $3q$ base arcs and $2q$ extra arcs in a component of size q. The network need not store the full dense matrix K_T or create T layers. For a vertex v, follow its orbit only until its first

repeat. This takes at most q distinct labels and determines a tail length μ, cycle length k, and entry phase. It gives $f^{T+1}(v)$ by a comparison and a remainder after the tail.

For the chosen cycle vertex v_0, let d be the first time the orbit of v reaches v_0, so $d < q$. Then the only nontrivial cost is

$$(K_T)_{v_0 v} = \begin{cases} 0, & T < d, \\ 1 + \lfloor (T-d)/k \rfloor, & T \geq d. \end{cases} \tag{19}$$

All costs are at most $T + 1$. All bounds are at most $\max(1, N)$, and the incidence balances have polynomial bit length in the original data. Even computing each orbit separately uses only polynomially many operations on integers of polynomial bit length. Integer lower bounds are handled by the usual shift into balances; loops and parallel arcs can be retained or processed by standard equivalent transformations.

Using any ordinary minimum-cost-flow algorithm with a polynomial bit-complexity guarantee, such as the established strongly polynomial network algorithm of Orlin [11], proves the claimed binary-input time bound. The compact certificate has $2n$ entries, each at most its corresponding m_y, so its output length is polynomial as well. Explicit word construction enumerates physical slots and protected suffixes. Its cost is polynomial in the expanded output size N and the input length; it need not be polynomial in the binary encoding of N alone. □

Remark 5.1 (Reduction versus reference implementation). The complexity theorem specifies a reduction to a polynomial algorithm for ordinary minimum-cost flow. The accompanying transparent exact reference program uses feasible-flow construction and negative-cycle cancellation. No polynomial binary-input worst-case runtime is claimed for that implementation. Its role is reproducible verification of the reduction and certificates. Replacing its flow backend by an established polynomial backend gives the theoretical bound.

6. Examples and precise limitations

6.1 The quota is about demand on the cycle

For $f = (1, 1)$, $T = 1$, $a = (1, 0)$ and $m = (1, 1)$, the word $(0, 1)$ works and $g = (0, 0)$, $h = (1, 0)$ is a certificate. The root has no demand and needs no forced endpoint. Requiring an endpoint at the root merely because the component has leaf demand would incorrectly reject it.

By contrast, for the identity map on two labels with $a = m = (1, 1)$ and $T = 1$, the first three certificate families admit $g = (1, 1), h = 0$, but no single host works. Those counts represent two closed cycles. The quotas exclude that false positive.

6.2 A fractional vertex is compatible with integer feasibility

Let

$$f = (1, 2, 0), \quad T = 1, \quad a = (1, 1, 1), \quad m = (3, 4, 4).$$

Then

$$K_1 = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \qquad \det K_1 = 2.$$

The real point $g = 0$, $h = (3/2, 5/2, 3/2)$ is feasible, with all capacity rows tight. Together with $g = 0$ these are six independent tight constraints, so the point is a fractional vertex. Summing capacities gives $\sum g + 2\sum h \leq 11$. Thus the maximum of $\sum h$ is $11/2$ over the real system and five over the integer system, the latter attained by $g = 0, h = (1, 2, 2)$. Consequently the polyhedron is not integral and arbitrary objectives need not have the same real and integer optimum. The instance still has an integer certificate. Its unequal capacities rule out the exact cycle shortcut.

6.3 A simple resource bound can miss the obstruction

For $f = (1, 2, 2)$, $T = 2$, $a = (4, 0, 2)$ and $m = (4, 4, 6)$, demand and matching give $h_0 + h_1 + h_2 \geq 4$. Root demand gives $g_2 + h_2 \geq 2$, and the root quota gives $h_2 \geq 1$. Hence root capacity must be at least

$$g_2 + h_0 + 2h_1 + 3h_2 \geq 2 + (h_0 + h_1 + h_2) + (h_1 + h_2) \geq 7.$$

Raising only m_2 to seven is sufficient, using $g = (1, 1, 1)$ and $h = (3, 0, 1)$. This is a resource obstruction even over the reals, not a rounding failure.

6.4 Scope of the unicyclic lemma

Consistent orientation of the unique cycle is essential. With two parallel arcs $J \to K$ and a twig $J \to L$, use

$$A = \begin{pmatrix} 2 & -1 \\ -1 & 2 \\ -1 & -1 \end{pmatrix}, \quad b = (0, 1, -1)^{\mathsf{T}}, \quad 0 \leq x \leq 1.$$

Both columns are signed walks from L to K and satisfy the incidence identity, but the unique underlying two-edge cycle is inconsistently oriented. The only real feasible point is $(1/3, 2/3)$, and no integer point exists. Its kernel direction has opposite signs on the parallel arcs.

One cycle is also essential for this argument. With two loops at J and an arc $J \to L$, take

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ -1 & -1 \end{pmatrix}, \quad b = (1, 1, -1)^{\mathsf{T}}, \quad 0 \leq x \leq 1.$$

The only real feasible point is $(1/2, 1/2)$. These columns again describe signed walks, but the kernel has two independent cycle directions. Neither example is a functional-map counterexample: a functional component has exactly one consistently oriented cycle.

Nor can an arbitrary aggregate inequality be appended to the lemma. On base arcs $J \to J, J \to K, J \to L$, let

$$A = \begin{pmatrix} 2 & 0 \\ 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad b = (1, 0, 0)^{\mathsf{T}}, \quad 0 \leq x \leq 1.$$

The unaugmented system is $x_1 = x_2 \leq 1/2$ and contains $(0, 0)$. Adding $x_1 + x_2 \geq 1$ leaves only $(1/2, 1/2)$. This explains why a quota requires its own argument; it does not refute the more structured functional-map quota conjecture.

6.5 The open aggregate implication

Let $\mathcal{R}(f, a, m, T)$ be the real polyhedron (4)–(7) with $g, h \geq 0$. The statement

$$\mathcal{R}(f, a, m, T) \neq \emptyset \implies \mathcal{R}(f, a, m, T) \cap \mathbb{Z}^{2n} \neq \emptyset \tag{20}$$

is not proved here for arbitrary functional maps. A demanded component is integer-feasible exactly when at least one forced real system $h_y \geq 1$ is feasible. It is not proved that an aggregate real solution must imply feasibility of any such forced system at a possibly different point. A positive total spread fractionally across several coordinates does not alone supply that conclusion.

For components whose target cycles are fixed points, the aggregate quota already is an individual bound, so Theorem 4.2 proves (20) for arbitrary incoming trees in that class. The earlier research record contains further special-class results; they are not dependencies or additional claims of this focused note. Resolving (20) is unnecessary for Theorem 1.1.

7. Relationship to established results

The model is a prescribed-letter-count circular-word problem. Its required words are $y, f(y), \ldots, f^T(y)$, with occurrence demands a_y. Cazaux and Rivals [7] use the same periodic-extension convention for circular occurrences. Their Multi-SCCS problem minimises the total length of a set of circles. It does not prescribe one circle with an exact inventory of letters. Crochemore et al. [8] study multiplicity superstrings, including length-two inputs and compact Eulerian-tour representations with binary multiplicities. Compact output for large multiplicities is therefore an established technique.

Cyclic scheduling supplies a close methodological precedent. Bartholdi, Orlin and Ratliff [3] reduce structured circular-ones integer programs to a bounded sequence of network problems and give special rounding procedures. Hochbaum and Levin [10] study covering, packing and mixed constraints whose rows are $0/1$ intervals in a linear or circular variable order. Following Veinott and Wagner, their partial-sum substitution turns an interval into a difference of two potentials; circular intervals introduce the total sum as one parameter. Their Sections 2, 4 and 10 develop this into parametric minimum-cost-flow and shortest-path algorithms. Individual variable bounds are included in their model, so bounds alone do not distinguish the present problem.

The matrix here has signed matching and demand rows, and orbit-visit coefficients as large as $T + 1$ when a trajectory winds repeatedly. Its incidence representation is on a consistently oriented unicyclic graph with attached trees. The crucial inequality uses the nonnegative cycle direction, and the one-circle conclusion additionally uses the reconstruction theorem. The cited interval results do not directly provide these hypotheses or conclusions. There is nevertheless a shared structure: complementing wrapping intervals writes a circular-ones matrix as a signed interval matrix plus a $0/1$ column times the all-ones row. Here (15) instead has an arbitrary integer row w. We know of no transformation reducing the full present model to their interval systems; differences in matrix form alone do not exclude one.

There is also an exact connection to affine TU decompositions. Equation (15) and total unimodularity of $[R\ z]$ give

$$A^{\mathsf{T}} = R^{\mathsf{T}} + w^{\mathsf{T}} z^{\mathsf{T}}, \qquad \begin{pmatrix} R^{\mathsf{T}} \\ z^{\mathsf{T}} \end{pmatrix} \text{ is totally unimodular.}$$

Thus A^{T} has affine TU dimension at most one in the sense of Bader et al. [2]. Gijswijt [9] establishes integer decomposition for the associated nearly TU integer hulls; adding a redundant zero row puts A^{T} into his definition. These results concern integer hulls in that orientation. They do not automatically transfer to real-to-integer feasibility of $Ax \leq b$. In particular, the row w in $A = R + zw$ is an arbitrary integer row, whereas the distinguished row in an affine TU decomposition must be $0, \pm 1$. The direct lifted proof in Section 3 explains the additional use of $z \geq 0$.

The bounded-subdeterminant framework of Aprile et al. [1] is another related use of network structure. It assumes a fixed bound on all subdeterminants and obtains a transposed network matrix after deleting a fixed number of rows and columns. Here the entries themselves can grow with the binary horizon, so that framework does not apply directly.

Incidence total unimodularity, integral bounded flows and polynomial minimum-cost-flow algorithms are inherited tools; Orlin [11] supplies the algorithmic dependency. The proposed contribution is the exact one-circle certificate, its signed-walk realisation, and the componentwise endpoint tests that give a binary-input polynomial decision without expanding trajectories. The comparisons identify these relationships without asserting historical priority for the specialised combination. Neither integral vertices nor arbitrary-objective integer optimisation follows from the feasibility theorem.

8. Connection to Occurrence Theory

8.1 From a qualitative question to an exact hosting problem

The originating research brief asked AI systems to seek interesting, potentially original mathematics motivated by Bartlett's independently developed *Something is happening: Occurrence Theory* (SiH) [4]. SiH distinguishes an abstract description from an occurring process and proposes that effective worlds may be realised through organised activity at underlying levels. Its bounded-base postulate gives a finite repertoire of complete states, while determinism is a further branch of the proposal rather than a requirement of the whole framework. Under the finite deterministic branch, a beginningless history lies on a recurrent cycle [4, Sections 2.3–2.4 and 9.1]. The host studied here represents one such complete-state cycle. It need not represent every state or every orbit of a larger base. On the finite deterministic branch, which SiH leans towards, this host could be the base cycle itself, one finite exact pass of the occurring base [4, Sections 9.3 and 9.5]. The higher-level trajectories would then be realised within that cycle, rather than necessarily forming separate cycles of their own. Figure 3 pictures this hosting relation.

SiH itself states the elementary exact-factor obstruction: if a finite recurrent update F is read by an onto coarse map π and satisfies $\pi F = f\pi$ everywhere on its recurrent domain, then f must be a permutation. Indeed, πF is onto, so f is onto, and an onto self-map of a finite set is invertible. That obstruction concerns an exact, autonomous law valid throughout the specified domain. It does not exclude finite-duration agreement from selected preparations. SiH locates apparent merging in an open subsystem, in hidden history that the complete state still carries, or in an approximation for selected preparations and durations [4, Sections 9.1 and 10.1].

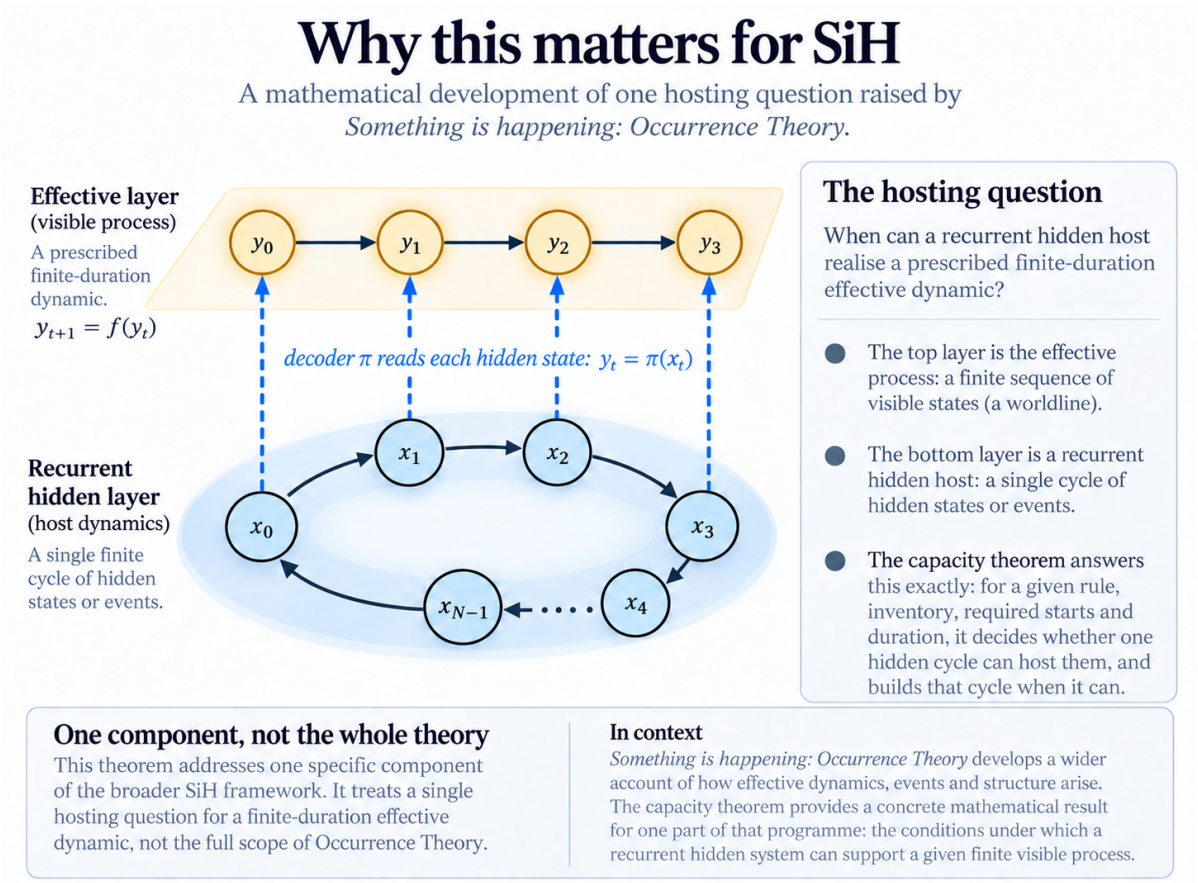


Figure 3: The hosting question. An effective process $y_0 \to y_1 \to \cdots$ obeys a visible rule, $y_{t+1} = f(y_t)$. A recurrent hidden host, one cycle of N complete states $x_0 \to x_1 \to \cdots \to x_{N-1} \to x_0$, realises it for T steps from x_0 when its decoder π reads $y_t = \pi(x_t)$ for $t = 0, \ldots, T$. This is the goodness condition $\pi(F^t x) = f^t(\pi x)$ of Section 1, drawn for $T = 3$. The capacity theorem decides exactly when a declared inventory and set of required starts can be hosted in this way for a given duration. It addresses one component of the hosting relation in *Something is happening*, not the whole theory.

The present theorem develops the case of selected preparations and durations in an exact form: within the promised windows there is no approximation error, while the distinctions the visible rule forgets remain in the host state. The correspondence is:

Mathematical object	Hosting interpretation
One N-position circle	One recurrent history of distinct complete host states.
Fixed visible labels	A decoder assigning an effective state to each host state.
Rule f	The deterministic effective law to be realised during a window.
Inventory m_y	The number of host states assigned the effective label y.
Demand a_y	The minimum number of distinct valid starting states with label y.
Horizon T	The number of effective transitions promised from each selected start.

Thus, in place of the qualitative observation that a finite recurrent carrier can accommodate such behaviour, the theorem gives an exact feasibility criterion for a declared inventory. It distinguishes allocations that suffice from allocations that cannot work, even with freely chosen ordering and decoding. The one-circle requirement matters because separately feasible histories on disconnected cycles need not all occur on the same recurrent history.

8.2 What this supplies to the reality-stack programme

The result supplies a trajectory-level component of a hosting relation: a fixed interpretation of one underlying process agrees with a specified effective update for every promised start and time. It also exposes a resource cost of demanding more starts or longer correct histories. A finite-duration effective law is not merely an inaccurate description: within its declared domain of operation it is exact, while omitting underlying distinctions that become relevant outside that domain.

This is narrower than SiH's Causal-Informational Parthood programme. Section 2.6 of SiH requires event-support maps, preservation of relevant causal organisation, an account of overlapping descriptions, temporal ordering and, for conscious subjects, further organisational criteria [4]. A state decoder and correct trajectory windows do not establish all those requirements. In particular, this paper does not preserve arbitrary interventions, establish the independence of several hosted worlds, or prove composition of event support across multiple levels. It compares one host transition with one effective transition; other timing interfaces require their own accounting.

The distinction from the computational companion *Zero Without Stasis* is also useful. That study investigates pattern formation under a selected local reversible signed-charge-conserving rule [5]. The present result instead characterises which finite trajectory requirements can be realised when the cyclic host is free to be constructed. It neither derives the ZWS rule nor explains its observed organisation. The two studies address different questions: organisation in a particular local dynamics, and exact hosting resources in a general but permissive finite model.

8.3 Remaining explanatory obligations

The effective rule, inventories and demands are inputs, not outcomes of evolution. The construction does not show that a simple base spontaneously produces its decoder, suitable preparations, repair mechanisms or higher-level worlds. It imposes no locality, noise resilience or preparation-complexity requirement. Counts of valid starts do not by themselves define an observer measure or establish typicality. Nor does the theorem derive primitive occurrence or consciousness.

These boundaries locate the contribution rather than detach it from its motivation. SiH raises the finite-duration hosting question; the certificate and algorithm answer a precise version of it. Future models could retain this as a permissive feasibility benchmark while adding operational interfaces, autonomous preparation and local implementation costs. Whether those stronger requirements can arise naturally remains a separate research question.

9. Applications and a maximum-horizon consequence

9.1 Exact-inventory periodic sequence synthesis

An immediate use is the literal synthesis problem of Section 1. A specification supplies label types Y, a single required successor rule f, an inventory m, a number a_y of acceptable starts of each type, and a required run length T. The algorithm either rejects the specification or returns a certificate from which a circular sequence can be built. No interpretation in terms of cosmology is needed.

A candidate application is the design of a periodic test-data sequence: several starting slots must expose specified deterministic continuations, while the total occurrences of each symbol are fixed. Here the output symbols are Y, the required continuation is f, and the inventory and start quotas are exactly m and a. The theorem synthesises the output sequence, not the entire testing apparatus. It does not handle arbitrary input-driven automata, prescribe named start positions, or ensure that a device can locate or select those positions. Additional such requirements would change the problem.

9.2 The longest duration supported by a fixed inventory

The decision theorem gives a useful consequence without a new integrality assumption.

Proposition 9.1 (Maximum feasible horizon)**.** *Fix a nonempty finite total map f and integer data $0 \le a \le m$ with $N = \sum_y m_y > 0$. If $a = 0$, or if alternative (i) of Theorem 2.1 holds, the requirements are feasible for every horizon. Otherwise there is a largest feasible horizon $T_{\max} \in \{0, \dots, N-1\}$. It can be found by binary search with at most $\lceil \log_2 N \rceil$ calls to the fixed-horizon decision procedure, after the two direct checks. Thus $T_{\max}$ is computable in time polynomial in the binary encoding of f, a, m.*

Proof. A word correct through horizon T is correct through every smaller horizon, with at least the same good starts. Feasibility is therefore downward closed in T. At $T = 0$ every position is good, so feasibility follows from $a \le m$. Zero demand is vacuous, and the exact-cycle alternative gives a word correct for all time. Otherwise $a \neq 0$, and no all-correct word has the required inventory, by the necessity part of the proof of Theorem 2.1. Any circle then has an incorrect edge. Every N-transition window traverses that edge, so $T = N$ is infeasible, as in Remark 2.2. Binary search between the known feasible endpoint 0 and known infeasible endpoint N needs at most $\lceil \log_2 N \rceil$ tests. All queried horizons have at most the bit length of N; applying Theorem 1.1 proves the complexity claim. The returned number is an optimum over possible hosts, not necessarily over histories of an independently prescribed host. □

As a worked design example, let $f(0) = f(1) = 1$, take four zero-labelled positions and seven one-labelled positions, and require two good zero-starts and one good one-start. Thus $m = (4, 7)$ and $a = (2, 1)$. Two distinct good zeros must each be followed by a run of at least T ones. A good one-start needs $T + 1$ consecutive ones. These either lengthen one of the two runs above to at least $T + 1$ ones, or form a further run. Consequently $7 \ge 2T + 1$. For $T = 3$ this is attained by

$$0\,1\,1\,1\,1\,0\,1\,1\,1\,0\,0 \quad \text{read cyclically.}$$

Its two zero-starts followed by long runs are good, as is the first one in the four-one run (Figure 4). Its certificate is $g = (1, 0)$, $h = (1, 1)$; then $K_3 h = (1, 7)$ and $g + K_3 h = (2, 7) \le (4, 7)$. At $T = 4$ the necessary bound would be $7 \ge 9$, which fails. The optimal horizon is therefore exactly three.

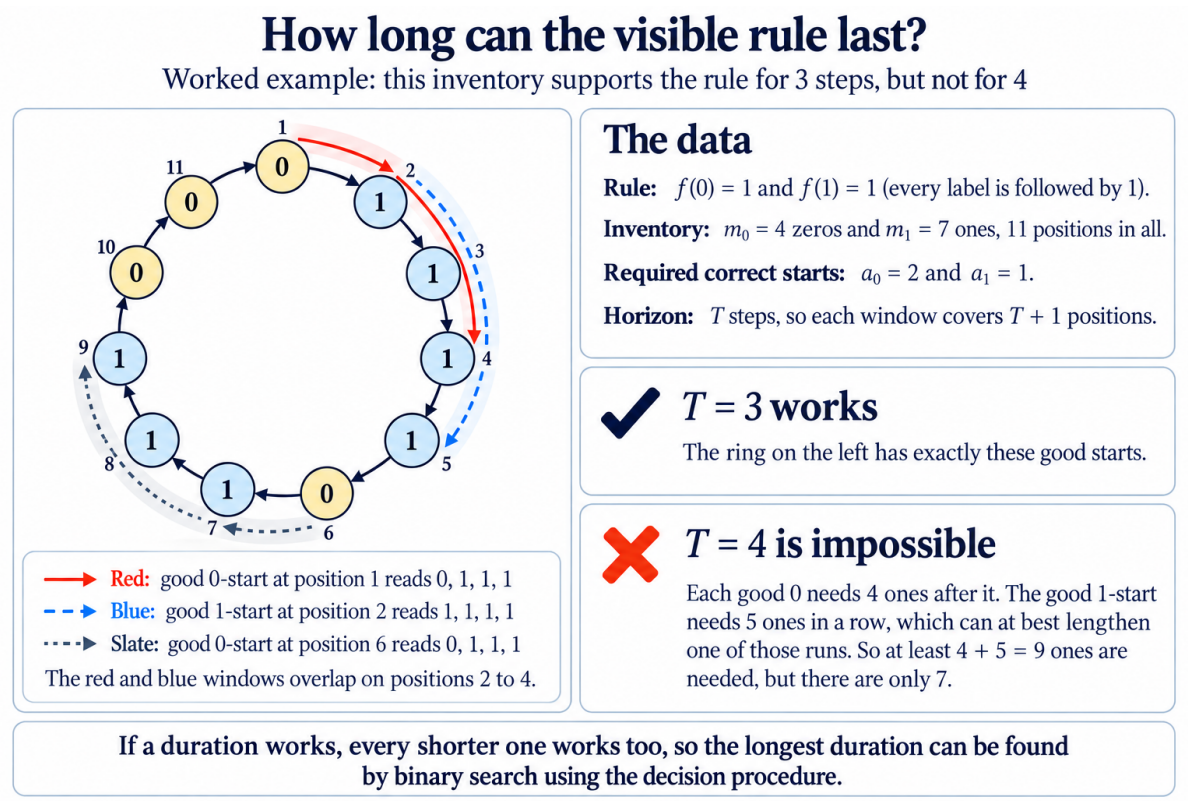


Figure 4: How long can the visible rule last? The worked design example: rule $f(0) = f(1) = 1$, inventory $m = (4, 7)$ and demands $a = (2, 1)$. At $T = 3$ the circular word $0\,1\,1\,1\,1\,0\,1\,1\,1\,0\,0$ (positions 1 to 11) has exactly two good 0-starts, at positions 1 and 6, and one good 1-start, at position 2; the windows from positions 1 and 2 overlap. At $T = 4$ each good 0 needs four ones after it, and the good 1-start needs five in a row, which can at best lengthen one of those runs: that is at least $2T + 1 = 9$ ones, but only seven exist. Because a duration that works makes every shorter one work, Proposition 9.1 finds the largest feasible horizon, here $T = 3$, by binary search.

The supplied demonstration runs the generic decision procedure and search for the maximum horizon, verifies the constructed word directly, and checks this boundary independently by enumerating all 330 binary words with four zeros and seven ones. These enumerated words are indexed sequences, not distinct classes up to rotation. The example is a mathematical demonstration, not a claim of deployed industrial use.

9.3 Benchmarks for finite reversible hosting

Reversible simulation of irreversible computation is established, including Bennett's reversible Turing-machine construction [6]. The present result does not replace that theory. It supplies an exact fixed-inventory decision for one finite cyclic host, a fixed decoder, selected starts and a finite horizon.

Within this same model, a negative answer is also an obstruction for any more restricted implementation whose hosted history is still one cycle of N complete states, with the same f, a, m, T and one-step timing: imposing locality or a restricted decoder cannot create a host where no unrestricted one exists. A positive answer has the opposite limitation: it need not give a local, efficient or fault-tolerant physical implementation. This makes the result a candidate baseline for studying the additional costs of such restrictions, rather than a ready-made hardware design.

The resource measured is the number of distinct complete states and their allocation to labels. A binary identifier for N alternatives needs at least $\lceil \log_2 N \rceil$ bits, but that counting fact does not specify a circuit, its gate count, its energy cost or the complexity of accessing valid preparations. An actual implementation must account for its controller and other state-bearing machinery. The reference code is suitable for exact finite experimentation; its flow backend is not claimed to meet the theorem's polynomial worst-case bound.

Future applications would need a concrete mapping to this interface and validation against any omitted requirements. In particular, the paper does not solve general scheduling, unrestricted automata conformance, or arbitrary-objective integer optimisation.

10. Reproducibility and verification scope

The archive deposited on Zenodo with this paper under DOI 10.5281/zenodo.23755207 contains the LaTeX source and figures, the reference implementation, an independently written verification suite, three Lean files, the demonstration of Section 9, and one script that reruns every check, with the results of that run. It also identifies the earlier versions of the paper that were reviewed. The seven theorem, lemma and corollary statements are those of the first version, for which the reference implementation was specified. Later versions clarified the proofs, notation and layout, added explanatory sections, examples and figures, and added the maximum-horizon proposition of Section 9, with a written proof and dedicated tests.

The reference implementation was developed from a mathematical specification separately from earlier solver implementations. Its tests compare network certificates with enumeration of physical circular words and integer certificates. The independently written Euler/circular-word suite, and a direct enumeration test of the explanatory examples and the maximum-horizon wrapper, check the same results by other routes. Input domains, seeds, counts, commands, environment and output hashes are recorded. Reproduction and PDF building run locally; neither cloud CI nor a remotely supplied randomness source is used.

Finite agreement tests implementations and examples; the arbitrary-size conclusions rest on the written proofs. The three Lean files formalise potential telescoping, the integer quota disjunction, and algebraic certificate recovery assuming the kernel identity. They do not formalise the single-circle construction, the kernel theorem, total-unimodularity integrality, the forced-endpoint theorem, complexity, or the maximum-horizon proposition. Their recorded builds are included; no fresh compilation is claimed for this version.

Acknowledgements and AI-assistance statement

This paper arose from a programme conceived by the author, based on his independently developed *Something is happening: Occurrence Theory* [4], where he identified the problem of reconciling finite recurrent dynamics with apparently irreversible higher-level processes. He directed AI systems to seek interesting, potentially original mathematics addressing it, and set the scope and stopping criteria. AI agents then defined the specific fixed-capacity circular-realisation problem, formulated and proved its theorems, wrote the software, ran the analyses and drafted the initial manuscript. The primary models were GPT-6-based, with contributions from Claude's Opus 5.5 and Fable 5.1 and xAI's Grok 4.6. The author reviewed stage summaries, approved or redirected investigations, commissioned critical reviews, and edited and approved the final manuscript. He deposited the release and accepts responsibility for its claims, interpretations and remaining errors. The author retains supporting project and review records; the release archive documents reviewed versions, checks and their scope.

11. Conclusion

In plain terms, a single repeating cycle of complete states can carry a visible process that forgets distinctions, for a specified number of steps and from at least the required number of starts, and the paper determines exactly when. Given the visible rule, the number of underlying states displaying each visible state, the required starts and the duration, Theorem 2.1 shows that a host exists exactly when the inventory is one constant exact cycle or two short lists of whole numbers satisfy four families of linear inequalities. Its proof builds the host from any such certificate, splicing separate loops into one circle without breaking the promised histories. By Theorem 1.1, existence is decidable in time polynomial in the binary length of the input, using at most one bounded minimum-cost-flow problem for each visible label, so a long duration need not be simulated, nor the circle written out, to answer the question.

The same test answers related design questions. For a fixed rule, inventory and demands, every duration works when the demands vanish or the inventory fits one cycle of the visible rule evenly; otherwise binary search finds the longest feasible duration (Proposition 9.1). A certificate also yields a periodic sequence with exact symbol counts and the required deterministic continuations (Section 9). For *Something is happening*, the result turns a qualitative question, whether a recurrent base can support effective dynamics that forget, into an exact hosting problem with a resource test; on the finite deterministic branch that SiH leans towards, the host could be the base cycle itself (Section 8).

Two directions remain open: the aggregate real-to-integer implication discussed in Section 6, and the additional cost of hosts restricted by locality or by their decoder, for which the exact test is a candidate baseline (Section 9). What was an illustrative possibility is now an exact test.

References

[1] Manuel Aprile, Samuel Fiorini, Gwenaël Joret, Stefan Kober, Michał Seweryn, Stefan Weltge, and Yelena Yuditsky. Integer programs with nearly totally unimodular matrices: The cographic case. *Mathematics of Operations Research*, 2026. Articles in Advance. https://doi.org/10.1287/moor.2024.0830.

[2] Jörg Bader, Robert Hildebrand, Robert Weismantel, and Rico Zenklusen. Mixed integer reformulations of integer programs and the affine TU-dimension of a matrix. *Mathematical Programming*, 169(2):565–584, 2018. https://doi.org/10.1007/s10107-017-1147-2.

[3] John J. Bartholdi, III, James B. Orlin, and H. Donald Ratliff. Cyclic scheduling via integer programs with circular ones. *Operations Research*, 28(5):1074–1085, 1980. https://doi.org/10.1287/opre.28.5.1074.

[4] Chris Bartlett. Something is happening: Occurrence Theory. Finitary Occurrent Actualism, Mathematical Intrinsic Occurrence, Evolutionary Cosmogenesis, and Conscious Actuality. Working paper, version 2.367, 13 October, 2026. https://doi.org/10.5281/zenodo.22875288.

[5] Chris Bartlett. Zero Without Stasis: Organisational Emergence in an Exactly Reversible Signed-Charge-Conserving Block Cellular Automaton. Companion working paper, version 1.49, 13 October, 2026. https://doi.org/10.5281/zenodo.22270456.

[6] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. https://doi.org/10.1147/rd.176.0525.

[7] Bastien Cazaux and Eric Rivals. Superstrings with multiplicities. In *29th Annual Symposium on Combinatorial Pattern Matching (CPM 2018)*, volume 105 of *Leibniz International Proceedings in Informatics*, pages 21:1–21:16, 2018. https://doi.org/10.4230/LIPIcs.CPM.2018.21.

[8] Maxime Crochemore, Marek Cygan, Costas Iliopoulos, Marcin Kubica, Jakub Radoszewski, Wojciech Rytter, and Tomasz Waleń. Algorithms for three versions of the shortest common superstring problem. In *Combinatorial Pattern Matching (CPM 2010)*, volume 6129 of *Lecture Notes in Computer Science*, pages 299–309, 2010. https://doi.org/10.1007/978-3-642-13509-5_27.

[9] Dion Gijswijt. Integer decomposition for polyhedra defined by nearly totally unimodular matrices. *SIAM Journal on Discrete Mathematics*, 19(3):798–806, 2005. https://doi.org/10.1137/S089548010343569X.

[10] Dorit S. Hochbaum and Asaf Levin. Optimizing over consecutive 1's and circular 1's constraints. *SIAM Journal on Optimization*, 17(2):311–330, 2006. https://doi.org/10.1137/040603048.

[11] James B. Orlin. A faster strongly polynomial minimum cost flow algorithm. *Operations Research*, 41(2):338–350, 1993. https://doi.org/10.1287/opre.41.2.338.

Working paper, version 1.32, 29 October 2026 · DOI 10.5281/zenodo.23755207